
Heterogeneous Temporal Graph Neural Networks for Forex Portfolio Allocation

Miguel Montes Lorenzo

Mathematical Engineering and Artificial Intelligence
Universidad Pontificia Comillas, Madrid, Spain
202105503@alu.comillas.edu

Abstract

Foreign exchange portfolio allocation is a suitable setting for heterogeneous temporal graph learning because currencies are relative assets linked to rates, commodities, equity risk, and macro-financial regimes. This project studies whether a Heterogeneous Temporal Graph Neural Network (HTGNN) can use those relations more effectively than pointwise neural baselines when allocating across USD, EUR, JPY, GBP, CNY, CAD, AUD, and CHF. The pipeline downloads daily Yahoo Finance series, groups instruments into market blocks, converts FX quotes into USD-denominated currency returns, and trains models to predict long-only allocation vectors rather than raw returns. The main modelling choice is to replace direct return prediction with supervised allocation imitation: each target is an ex-post mean-variance teacher portfolio built from the next realised cross-section and recent covariance matrix. The HTGNN extends the signal-only setup by encoding each market block as a temporal node with a GRU and applying relation-typed message passing centred on the portfolio signal node. The reported backtests compare the graph model with a pointwise NN and simple currency benchmarks.

1 Introduction and Research Question

Foreign exchange (Forex or FX) is inherently relative: each currency is priced against another numeraire. This project fixes USD as numeraire and allocates across $\mathcal{C} = \{\text{USD}, \text{EUR}, \text{JPY}, \text{GBP}, \text{CNY}, \text{CAD}, \text{AUD}, \text{CHF}\}$. The prediction target is therefore not a price, but a long-only portfolio $w_t \in \Delta_N$, with $\Delta_N = \{w \in \mathbb{R}_+^N : \mathbf{1}^T w = 1\}$.

The research question is: **can a heterogeneous temporal graph neural network use macro-financial market blocks to improve currency allocation over signal-only neural baselines?** The hypothesis is that exchange rates are partly shaped by structured relations among currency returns, rates, commodities, equity risk, and FX futures. Encoding these sources as typed nodes and relations should fit the problem better than a pointwise MLP that only observes past portfolio signals.

1.1 Why Forex is an Interesting Prediction Domain

Forex is attractive for this project because currencies are macro assets. Their returns are linked to monetary policy, inflation, external balances, commodity exposure, and global risk appetite, which can be organised into meaningful market blocks. The task is also naturally cross-sectional: the model allocates among currencies under a common numeraire rather than forecasting isolated price levels.

The target universe is compact and interpretable. With eight currencies, one can inspect whether the model shifts capital toward plausible regimes, such as USD and CHF under stress or AUD and CAD in commodity-sensitive environments. The main limitation is that currencies lack the long-run growth story of equities: the problem is mostly relative value, exposed to inflation, policy expectations, central-bank intervention, and geopolitical shocks.

1.2 Why an HTGNN is Appropriate

A standard temporal model treats inputs as one homogeneous sequence, but the data here are grouped by economic role: FX pairs, FX futures, Treasury futures, bond ETFs, commodities, and equity-index futures. A heterogeneous graph preserves that structure while allowing information to pass through learned relation types.

Temporality is also necessary because allocation depends on recent paths. In the implemented HTGNN, each market block receives a return window, is encoded by a GRU, and then participates in typed message passing centred on the `portfolio_signal` node. The final allocation is read from that node, matching the intended design: market blocks provide context, and the portfolio node produces the currency weights.

1.3 Related Work and Scope

The project belongs to the heterogeneous and temporal GNN setting. Relational GNNs introduce relation-specific transformations for typed edges, making them a natural reference for heterogeneous financial graphs [1]. Temporal graph networks motivate the idea that node representations should be updated with time-dependent information rather than treated as static embeddings [2]. The implemented model is simpler than a full continuous-time TGN, but it follows the same high-level idea: temporal node states are built before graph aggregation.

The allocation target is based on mean-variance portfolio theory, where a portfolio is chosen by trading expected return against variance [3]. The repository also includes a Black-Litterman-style allocation strategy that can blend prior equilibrium returns with model views [4], but the main report focuses on the deterministic signal and HTGNN allocation pipeline.

2 Data Structuring

The repository uses daily adjusted Yahoo Finance price series from 2017-01-01 to 2026-01-01. Prices are converted to log returns, short gaps are forward-filled with a limit of three days, and instruments with excessive missingness are filtered before tensor construction. The supervised dataset uses a 20-day lookback window and chronological splits: 2018-01-01 to 2024-01-01 for training, 2024-01-01 to 2025-01-01 for validation, and 2025-01-01 to 2026-01-01 for test.

The target basket contains eight currencies. For every non-USD currency, the Yahoo Finance quote `USDXXX=X` is interpreted as units of currency XXX per one USD. Therefore the USD-denominated log return of holding one unit of currency XXX has the opposite sign of the quoted pair return:

$$r_{t, \text{XXX}}^{\text{USD}} = \log \left(\frac{V_t^{\text{XXX}}}{V_{t-1}^{\text{XXX}}} \right) = - \left(\log P_t^{\text{USD/XXX}} - \log P_{t-1}^{\text{USD/XXX}} \right), \quad r_{t, \text{USD}}^{\text{USD}} = 0. \quad (1)$$

This convention makes all target returns comparable under the same numeraire. The model output is then an allocation across the eight USD-valued currency returns.

2.1 Yahoo Finance Market Blocks

The heterogeneous graph does not create one node per individual ticker. Instead, each node represents a market block. This keeps the graph compact and forces the model to learn relations between economic categories rather than between a large set of weakly interpretable individual symbols.

Block	N	Examples	General relevance for Forex
commodity_future	22	GC=F, SI=F, CL=F, BZ=F, NG=F, ZC=F	Commodity prices proxy terms-of-trade shocks, inflation pressure, and risk regimes relevant to commodity-linked currencies such as AUD and CAD.
commodity_etf	12	GLD, IAU, SLV, USO, UNG, DBC, GSG	ETF versions provide tradable commodity exposure and can be less sparse than some futures histories.
equity_index_future	4	ES=F, NQ=F, YM=F, RTY=F	Equity-index futures proxy global risk appetite, which affects safe-haven and cyclical currency flows.
us_treasury_future	4	ZB=F, ZN=F, ZF=F, ZT=F	Treasury futures summarize changes in the US rates curve, a central driver of USD strength and carry conditions.
fx_future	6	6E=F, 6B=F, 6J=F, 6S=F, 6A=F, 6C=F	Currency futures provide alternative FX-market information for liquid developed-market currencies.
us_bond_etf	20	SGOV, SHV, SHY, IEF, TLT, AGG, LQD, HYG, TIP	US bond ETFs capture duration, credit, inflation-protected, and risk-on/risk-off fixed-income regimes.
fx_usd_pair	7	USDEUR=X, USDJPY=X, USDCAD=X, USDBCHF=X, USDAUD=X, USDBCHF=X	The direct FX quotes used to build the target currency-return basket.
eur_bond_etf	4	CBE3.L, SDEU.L, SYBV.DE, 020Y.L	Euro-area government-bond proxies reflect regional rate and duration expectations affecting EUR.
jpy_bond_etf	3	236A.T, CEB2.DE, 2510.T	Japan bond proxies provide information about JPY rate conditions and safe-haven behaviour.
gbp_bond_etf	9	IGLT.L, IGLS.L, INXG.L, VG0V.L, GLTY.L, GLTL.L	UK gilt proxies encode sterling-specific rate and inflation-linked bond information.
cny_bond_etf	13	2829.HK, CNYB.AS, CGB.L, ICGB.DE, CBGB.L	China bond proxies provide context for CNY and emerging-Asia rate conditions.
cad_bond_etf	5	XGB.TO, CLF.TO, CLG.TO, ZGB.TO, VGV.TO	Canadian government-bond proxies support CAD allocation through local-rate information.
aud_bond_etf	8	IGB.AX, VGB.AX, AGVT.AX, ALTB.AX, XGOV.AX	Australian bond proxies support AUD allocation and commodity-sensitive macro regimes.
chf_bond_etf	7	CSBGC3.SW, CSBGC7.SW, CSBGC0.SW, SB1CHA.SW, SB7CHA.SW	Swiss bond proxies give CHF-specific duration and safe-haven context.

Table 1: Yahoo Finance symbols grouped into heterogeneous market blocks. The number column is the configured number of tickers before transform-time missing-data filtering.

2.2 Tensor Construction

After filtering and alignment, every sample contains a lookback window ending at t and a next-day return vector at $t + 1$. The pointwise signal is the currency return window, later optionally converted inside the model into a sequence of mean-variance allocation features. The graph input is a dictionary of node windows:

$$X_t = \{X_{v,t-L+1:t} : v \in \mathcal{V}\}, \quad X_{v,t-L+1:t} \in \mathbb{R}^{L \times F_v}, \quad L = 20. \quad (2)$$

The metadata stores the list of node names, the input dimension of each node, the target currencies, the date splits, and the train-only standardisation statistics. Standardisation is fitted on the training split and then applied to all splits, avoiding validation or test leakage.

3 Signal Structuring: Learning Allocations Instead of Returns

The main methodological choice in the repository is to avoid treating the task as direct next-return regression. Daily financial returns have low signal-to-noise ratios, and a model trained to predict each return independently can learn unstable point forecasts that are not aligned with the final decision. The final use of the model is not to report tomorrow’s EUR or JPY return; it is to decide how much capital should be allocated to each currency. For this reason, the training signal is constructed in allocation space.

This design has three practical advantages. First, the output is constrained to the long-only simplex, which prevents nonsensical unconstrained predictions. Second, the target incorporates risk through a covariance matrix, not only through expected return. Third, the supervised target is closer to the downstream objective: a portfolio allocation. The model is therefore trained to imitate the kind of decision a portfolio optimiser would make, rather than to solve a harder and more weakly identified forecasting problem as an intermediate step.

3.1 Mean-Variance Teacher

Let $\mathbf{r}_t \in \mathbb{R}^N$ be the USD-denominated log-return vector of the eight currencies. At date t , the recent return window is $R_t = [\mathbf{r}_{t-L+1}, \dots, \mathbf{r}_t]$. The covariance estimator used by the training loss is

$$\hat{\Sigma}_t = \frac{1}{L-1} \sum_{i=t-L+1}^t (\mathbf{r}_i - |(\mathbf{r})_t|)(\mathbf{r}_i - |(\mathbf{r})_t|)^T + \varepsilon \mathbf{I}, \quad (3)$$

where ε is the ridge term. The target mean proxy is the realised next cross-section centred around its own average:

$$\tilde{\boldsymbol{\mu}}_{t+1} = \mathbf{r}_{t+1} - \frac{1}{N} \mathbf{1} \mathbf{1}^T \mathbf{r}_{t+1}. \quad (4)$$

The ex-post teacher allocation is then obtained from a long-only mean-variance problem:

$$\mathbf{w}_{t+1}^* = \operatorname{argmax}_{\mathbf{w} \in \Delta_N} \mathbf{w}^T \tilde{\boldsymbol{\mu}}_{t+1} - \frac{\gamma}{2} \mathbf{w}^T \hat{\Sigma}_t \mathbf{w}. \quad (5)$$

The unconstrained solution is proportional to $(\gamma \hat{\Sigma}_t)^{-1} \tilde{\boldsymbol{\mu}}_{t+1}$. Since the project uses long-only portfolios, this raw solution is projected onto the simplex:

$$\mathbf{w}_{t+1}^* = \Pi_{\Delta_N} \left((\gamma \hat{\Sigma}_t)^{\dagger} \tilde{\boldsymbol{\mu}}_{t+1} \right). \quad (6)$$

This does not imply that the model has access to $\mathbf{r}_{t+1}$ at inference time. The next return is used only to build the supervised teacher during training. At inference, the trained model receives only past windows and graph node features and outputs an allocation estimate.

3.2 Input Signal as Rolling Allocation History

The pointwise models can receive raw currency-return windows, but the selected configuration uses `input_format: "weights"`. In that mode, the model converts each prefix of the 20-day return window into a rolling mean-variance weight vector. The input becomes a sequence of previous allocation regimes rather than a raw return matrix:

$$\mathbf{X}_t^{\text{portfolio}} = [\mathbf{w}_{t-L+1}^{\text{in}}, \dots, \mathbf{w}_t^{\text{in}}], \quad \mathbf{w}_s^{\text{in}} = \Pi_{\Delta_N} \left((\gamma \hat{\Sigma}_{s-1})^{\dagger} \tilde{\boldsymbol{\mu}}_s \right). \quad (7)$$

The first step has insufficient history and is set to equal weights. This input representation is less sensitive to raw return scale and more directly aligned with the output. It also makes the comparison between the pointwise NN and HTGNN conceptually clear: the pointwise baseline sees only the allocation signal, whereas the HTGNN sees that same signal plus heterogeneous contextual nodes.

3.3 Dirichlet Allocation Head

The implemented models output Dirichlet concentration parameters rather than unconstrained weights. If $\mathbf{a}_t$ denotes the raw output logits, the concentrations and predicted allocation are

$$\boldsymbol{\alpha}_t = \text{softplus}(\mathbf{a}_t) + \varepsilon, \quad \hat{\mathbf{w}}_t = \frac{\boldsymbol{\alpha}_t}{\mathbf{1}^T \boldsymbol{\alpha}_t}. \quad (8)$$

This is useful even for deterministic models, because the mean of a Dirichlet naturally lies on the simplex. For stochastic models, the same parameterisation also provides a distribution over allocations. The training loss selected in the configuration is a sliced Wasserstein loss between samples from the predicted Dirichlet and samples from a target Dirichlet centred at the teacher allocation:

$$\mathcal{L}_{\text{SW}} = \frac{1}{M} \sum_{m=1}^M W_1 \left(\{\theta_m^T \mathbf{u}_i\}_{i=1}^S, \{\theta_m^T \mathbf{v}_i\}_{i=1}^S \right), \quad (9)$$

where $\mathbf{u}_i \sim \text{Dir}(\boldsymbol{\alpha}_t)$, $\mathbf{v}_i \sim \text{Dir}(\boldsymbol{\alpha}_t^*)$, and θ_m are random one-dimensional projection directions. This objective compares distributions in allocation space rather than applying a simple MSE to weights.

3.4 Why This is Preferable to Direct Return Prediction

A direct-return model would learn a map $f_{\theta(X_t)} \rightarrow \hat{\mathbf{r}}_{t+1}$ and then require a separate optimiser to transform predicted returns into weights. This creates a mismatch: small errors in noisy return forecasts can be amplified by the inverse covariance matrix and produce unstable allocations. In contrast, the implemented pipeline learns the allocation mapping directly. The model is penalised for placing capital in the wrong part of the simplex, which is closer to the final portfolio objective.

This is especially relevant for Forex. The differences between daily currency returns are often small, and the sign of tomorrow's return is difficult to predict. However, allocation labels can still encode useful relative information: which

currency was favoured after adjusting for recent covariance and after centering the realised cross-section. The model therefore learns a smoothed portfolio decision boundary rather than a set of isolated scalar forecasts.

4 Asset Allocation Strategies

The repository separates model prediction from portfolio rebalancing. A model outputs a target allocation $\hat{\mathbf{w}}_t$. A strategy then decides which portfolio is actually held. Three strategies are implemented.

Full rebalancing applies the model target directly:

$$\mathbf{w}_t^{\text{hold}} = \hat{\mathbf{w}}_t. \quad (10)$$

This is the cleanest strategy for comparing model outputs, but it can imply high turnover because the portfolio fully adapts to every daily prediction.

Partial rebalancing moves only a fraction α of the way from the current portfolio to the model target:

$$\mathbf{w}_t^{\text{hold}} = \text{normalize}(\mathbf{w}_{t-1}^{\text{hold}} + \alpha(\hat{\mathbf{w}}_t - \mathbf{w}_{t-1}^{\text{hold}})). \quad (11)$$

The default configuration uses $\alpha = 0.2$. This is more conservative and can reduce turnover, but it also delays adaptation when the model detects a new regime.

Black-Litterman rebalancing is available for probabilistic models. It treats the model output as a view and uses model uncertainty to scale the view-noise matrix. With an equilibrium prior $\mathbf{\Pi} = \delta \Sigma \mathbf{w}^{\text{eq}}$, view matrix $P = I$, view vector $\mathbf{q} = \hat{\mathbf{w}}_t$, and uncertainty matrix Ω , the posterior mean is

$$\boldsymbol{\mu}_{\text{BL}} = [(\tau \Sigma)^{-1} + P^T \Omega^{-1} P]^{-1} [(\tau \Sigma)^{-1} \mathbf{\Pi} + P^T \Omega^{-1} \mathbf{q}]. \quad (12)$$

The final strategy then applies a long-only mean-variance step to $\boldsymbol{\mu}_{\text{BL}}$ and the posterior-adjusted covariance. This strategy is not the central comparison for the deterministic HTGNN, but it is useful as an extension point for future uncertainty-aware graph models.

5 Comparative Baseline

One pointwise neural baseline is implemented for comparison with the HTGNN: a deterministic neural network (NN). It receives only the portfolio signal. In the selected configuration, that signal is the sequence of rolling mean-variance allocations derived from the past currency return window. It does not receive the additional heterogeneous market blocks.

5.1 Deterministic NN

The NN baseline is a multilayer perceptron. It flattens the 20-day portfolio signal and maps it through BatchNorm, GELU activations, dropout, and linear layers:

$$\mathbf{z}_t = \varphi_{\theta}(\text{vec}(\mathbf{x}_t^{\text{portfolio}})), \quad \boldsymbol{\alpha}_t = \text{softplus}(W_{\alpha} \mathbf{z}_t + \mathbf{b}_{\alpha}) + \varepsilon. \quad (13)$$

The predicted allocation is the Dirichlet mean $\hat{\mathbf{w}}_t = \frac{\boldsymbol{\alpha}_t}{\mathbf{1}^T \boldsymbol{\alpha}_t}$. This baseline tests how much can be learned from the allocation signal alone, without graph structure.

5.2 Fairness of the Comparison

The comparison is not fully fair if interpreted only as a model-architecture comparison, because the HTGNN receives more information than the NN. The pointwise baseline sees only the portfolio signal, while the graph model also receives commodities, futures, bond ETFs, FX pairs, and currency-region bond proxies.

However, the comparison remains informative from a practical financial-prediction perspective. Many signal models are trained only on the historical signals they later predict or rank. Under that convention, the pointwise model represents a realistic signal-only baseline. The HTGNN then answers a more specific question: does adding structured heterogeneous market context improve the allocation pipeline beyond a signal-only neural model?

6 Heterogeneous Temporal Graph Neural Network

The HTGNN is the main geometric deep learning model in the project. It models a fixed heterogeneous graph whose nodes are market blocks and whose edges are typed according to their economic role. The portfolio signal is one node in this graph.

Other nodes provide contextual information: FX-market nodes, rate-market nodes, commodity-market nodes, equity-risk nodes, and a small number of learned generic latent nodes.

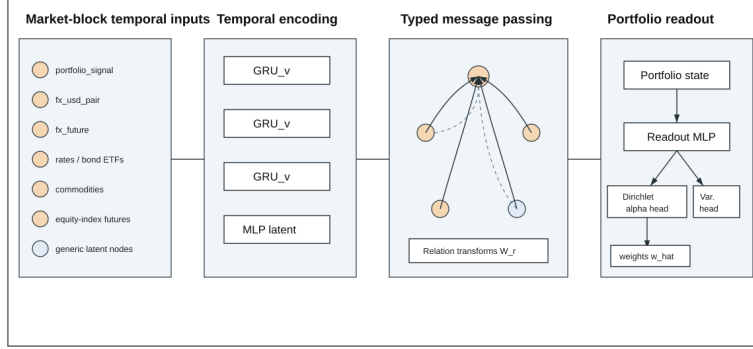


Figure 1: Implemented HTGNN architecture. Each market block is encoded as a temporal node, relation-specific message passing is applied over a portfolio-centred heterogeneous graph, and the final allocation is read from the portfolio node.

6.1 Temporal Node Encoding

For each observed node v , the input is a window $X_{v,t-L+1:t} \in \mathbb{R}^{L \times F_v}$. Since each market block has a different number of symbols, each node has its own GRU input dimension but all nodes are mapped into the same hidden size H :

$$h_v^0 = \text{GRU}_{v(X_{v,t-L+1:t})} \in \mathbb{R}^H. \quad (14)$$

This temporal encoder compresses the recent behaviour of a whole market block into a single state. For example, the commodity_future node receives the returns of commodity futures over the lookback window and maps them into one hidden state; the us_bond_etf node does the same for the US fixed-income ETF block.

The implementation also supports learned generic latent nodes. These nodes do not correspond to observed Yahoo symbols. They are trainable embeddings that can act as global latent factors or information-routing nodes. In the selected HTGNN configuration, three generic latent nodes are included.

6.2 Heterogeneous Message Passing

After temporal encoding, the model applies relation-typed message passing. For a target node v , incoming messages are transformed according to their relation r :

$$m_v^\ell = \frac{1}{|\mathcal{N}(v)|} \sum_{(u,v,r) \in \mathcal{E}} W_r^\ell h_u^\ell. \quad (15)$$

The node update combines the previous node state with the aggregated message and uses a residual layer-normalised update:

$$h_v^{\ell+1} = \text{LayerNorm}(h_v^\ell + \text{MLP}_{\ell}([h_v^\ell; m_v^\ell])). \quad (16)$$

The important detail is that W_r depends on the edge type. A message from FX pairs to the portfolio node is not forced to use the same transformation as a message from bond ETFs or commodities. The relation names in the implementation include fx_to_portfolio, rates_to_portfolio, commodity_to_portfolio, equity_to_portfolio, portfolio_to_market, market_cross, and analogous relations involving generic latent nodes.

6.3 Portfolio Readout

Only the final portfolio_signal state is used for prediction. After K message passing layers,

$$z_t = \rho(h_{\text{portfolio}}^K), \quad \alpha_t = \text{softplus}(W_\alpha z_t + b_\alpha) + \varepsilon, \quad (17)$$

$$\hat{w}_t = \frac{\alpha_t}{1^T \alpha_t}, \quad \hat{\sigma}_t^2 = \text{softplus}(W_\sigma z_t + b_\sigma) + \varepsilon. \quad (18)$$

The readout therefore predicts both the allocation and a positive variance estimate. The selected configuration uses hidden dimension 96, one GRU layer, two message-passing layers, GELU activations, dropout 0.10, batch size 64, and a sliced Wasserstein allocation loss.

6.4 Difference from a Simple GNN: Heterogeneity

A simple GNN usually assumes a homogeneous graph: every node has the same feature semantics and every edge applies the same message function. This is not well matched to the present data. A commodity-futures node, a bond-ETF node, and an FX-pair node do not represent the same kind of information. Their relation to the portfolio signal should be transformed differently.

The HTGNN uses heterogeneity in two places. First, each node has a different input feature dimension because each market block contains a different number of Yahoo Finance symbols. Second, edges have relation types, and each relation type has its own linear message transform. This is the part that makes the graph model economically interpretable: rates-to-portfolio, FX-to-portfolio, commodity-to-portfolio, and equity-to-portfolio are different learned channels.

6.5 Difference from a Simple GNN: Temporality

A simple static GNN would consume one graph snapshot at one date. The model here is temporal because each node contains a 20-day sequence. The GRU encoders allow recent path information to affect the node state before graph aggregation. This matters in financial data because the meaning of today's return often depends on recent trend, reversal, volatility, and co-movement patterns.

The model is not a continuous-time event graph and does not update memory after every trade. It is a daily-window temporal graph model. This is appropriate for the available data frequency and for the portfolio horizon considered in the repository.

6.6 How the HTGNN Uses the Project-Specific Structure

The implemented graph is portfolio-centred. Directed edges are created when the source or target is the `portfolio_signal` node, or when a generic latent node is involved. This design avoids an unnecessarily dense fully connected market graph while still allowing every contextual block to influence the final portfolio state. Market-to-market interactions can also occur through generic latent nodes and through relation types such as `market_cross` when allowed by the edge builder.

This structure matches the financial interpretation of the task. The model does not need to predict every market block; it needs to transform those blocks into a currency allocation. The portfolio node therefore acts as the central readout node, while other nodes act as typed sources of macro-financial context.

7 Results

The analysis compares the deterministic HTGNN with a deterministic signal-only NN. Bayesian variants are omitted so the main question stays clear: does heterogeneous temporal market context improve allocation?

7.1 HTGNN Performance Against Backtest Baselines

Over 2024-01-01 to 2025-12-31, the HTGNN separates from the reference backtest baselines. With direct full rebalancing, it reaches 265.7% cumulative return and 87.0% annualized return.

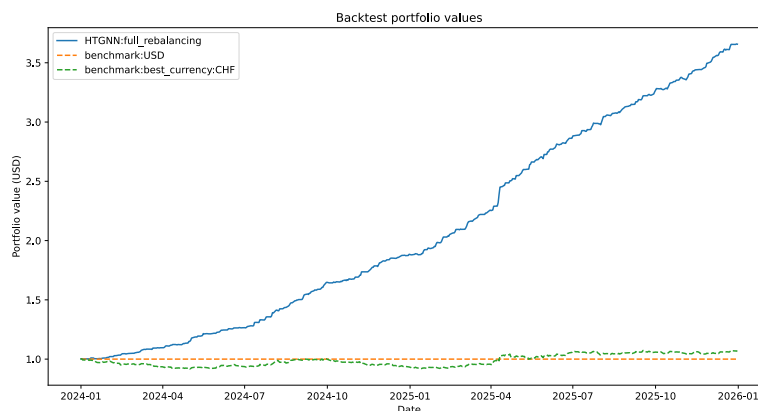


Figure 2: Backtest comparison between the deterministic HTGNN and three reference comparators: the deterministic NN baseline, 100% USD, and 100% CHF as the hindsight-best single-currency benchmark.

The performance does not come from collapsing into one or two currencies. AUD (23.0%), JPY (17.3%), and USD (15.9%) are largest on average, but CNY, GBP, CHF, and CAD also contribute; EUR remains secondary at 3.1%.

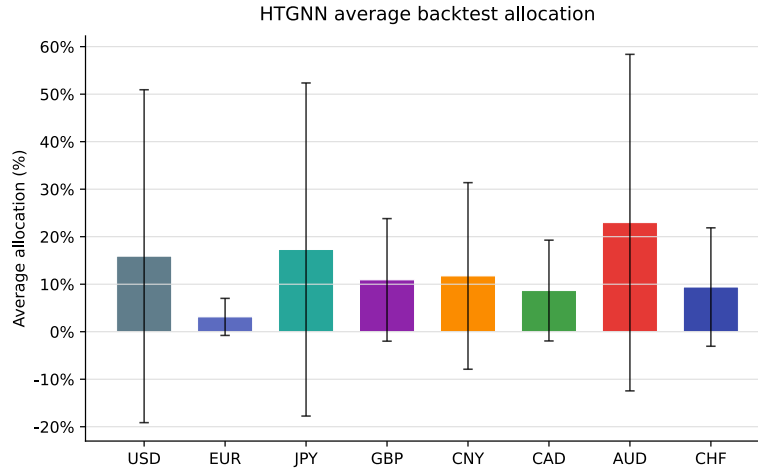


Figure 3: Average backtest allocation of the deterministic HTGNN. The portfolio is not concentrated permanently in one or two currencies, but spread across several relevant exposures.

Figure 2 and Figure 3 show a profitable but still diversified allocation, rather than a single-currency allocation.

7.2 HTGNN Versus the Pointwise NN

The direct research comparison is HTGNN versus signal-only NN. The graph model outperforms the baseline.

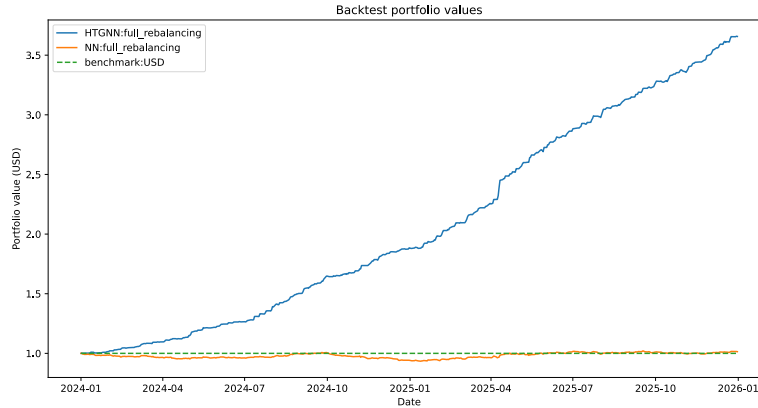


Figure 4: Backtest comparison between the deterministic HTGNN, the deterministic NN baseline, and the 100% USD baseline.

The NN baseline ends with only 1.4% cumulative return and 0.68% annualized return, barely above 100% USD. The gap suggests that heterogeneous context is material for this FOREX allocation problem.

The final comparison reports performance metrics and allocation-stability metrics for HTGNN, NN, and the 100% USD benchmark.

System	Cum. ret.	Ann. ret.	Ann. vol.	Sharpe	Sortino	Max DD
HTGNN	265.7%	87.0%	6.76%	9.26	40.82	-0.64%
NN	1.4%	0.68%	4.90%	0.14	0.22	-7.34%
100% USD	0.0%	0.0%	0.00%	0.00	0.00	0.00%

Table 2: Performance metrics on the evaluation period. Bold marks the best value in each column, with lower values preferred for annualized volatility and maximum drawdown.

gesting that broad FX, fixed-income, and commodity regime signals carry more useful structure here than local treasury proxies alone.

8 Conclusions

The results indicate that FOREX allocation benefits from heterogeneous temporal graph learning. The deterministic HTGNN outperforms the signal-only NN, suggesting that FX futures, commodity blocks, and fixed-income blocks add useful predictive context.

The graph model also avoids simply copying temporal persistence: its lag-1 allocation structure remains closer to the weakly correlated input signal than to the persistent NN outputs. This fits a cross-asset FOREX setting, where useful information is distributed across markets.

The node-focus analysis adds interpretability. The HTGNN relies most on FX futures, broad US fixed income, and commodity-linked nodes, while using local sovereign-bond proxies less. This is consistent with a setting where broad cross-asset regime signals matter more than local bond proxies alone.

9 Future Work

One extension is uncertainty-aware allocation. Financial inputs are incomplete and non-stationary, so a deterministic HTGNN can produce strong point allocations but cannot express confidence directly.

A Bayesian HTGNN is therefore worth testing. The repository already contains an MC-dropout BHTGNN, but it was not validated enough for the main comparison. Future work should test deterministic HTGNN, MC-dropout BHTGNN, and possibly a variational graph model under the same splits and trading rules.

The key question is empirical: whether uncertainty estimates improve realized portfolio quality after turnover and risk are included. In FOREX, observable macro drivers may already capture much of the relevant structure, so Bayesian modelling remains an empirical question rather than a guaranteed improvement.

References

- [1] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, “Modeling Relational Data with Graph Convolutional Networks,” in *The Semantic Web: 15th International Conference, ESWC 2018*, Springer, 2018, pp. 593–607.
- [2] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, “Temporal Graph Networks for Deep Learning on Dynamic Graphs,” in *ICML Workshop on Graph Representation Learning*, 2020.
- [3] H. Markowitz, “Portfolio Selection,” *The Journal of Finance*, vol. 7, no. 1, pp. 77–91, 1952.
- [4] F. Black and R. Litterman, “Global Portfolio Optimization,” *Financial Analysts Journal*, vol. 48, no. 5, pp. 28–43, 1992.